自然語言處理與應用
Natural Language Processing and Applications

Hugging Face Tutorial

Instructor: 林英嘉 (Ying-Jia Lin)
2026/04/27

GitHub

Slido
NLP0427
(Link)

What you will learn in this tutorial

- Training and evaluating GPT-2 / T5 on abstractive summarization
  - Dataset: Chinese abstractive summarization
  - Objective: Cross-entropy
  - Main packages: PyTorch, Hugging Face, ROUGE

Why do we need to learn GPT-2?

- GPT-2 is the **last** open-source model in the GPT series from OpenAI.
- Language generation is hard, and GPT-2 is a good start point.
- GPT-2 is not big. It is feasible for low-budget machines.
  - GPT-2 (12-layer; 124M), GPT-2-medium (24-layer; 345M), GPT-2-large (36-layer; 762M), GPT-2-xl (48-layer; 1.5B)

Introduction to Text Summarization

- Extractive summarization
  - Generate a short text summary for a document by selecting salient sentences in the documents.
- Abstractive summarization
  - Generate novel words and phrases not featured in the source text.

Example:
The Queen's Birthday holiday road toll stands at zero for the first time since records began.

Ext summary: The Queen's holiday road toll stands at zero.
Abs summary: The Queen's holiday slashes road toll.

Task: Chinese Abstractive Summarization

- Dataset: LCSTS (A Large-Scale Chinese Short Text Summarization Dataset)[1]

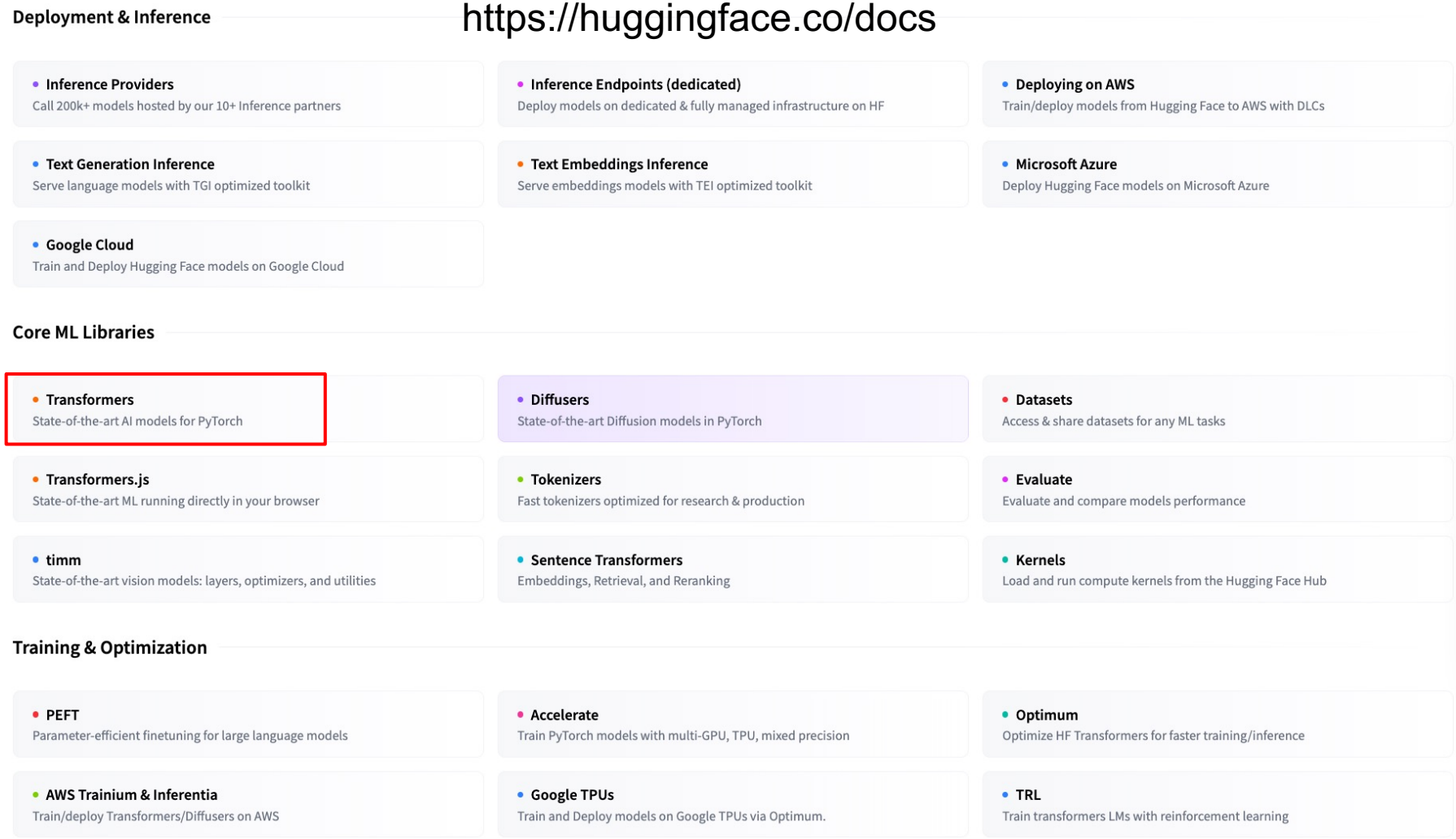
Short Text: 水利部水资源司司长陈明忠今日在新闻发布会上透露，根据刚刚完成的水资源管理制度的考核，有部分省接近了红线的指标，有部分省超过红线的指标。在一些超过红线的地方，将对一些取用水项目进行区域的限批，严格地进行水资源论证和取水许可的批准。

Summarization: 部分省超过年度用水红线指标 取水项目将被限批

[1] Hu, Baotian, Qingcai Chen, and Fangze Zhu. "LCSTS: A Large Scale Chinese Short Text Summarization Dataset." Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing. 2015.

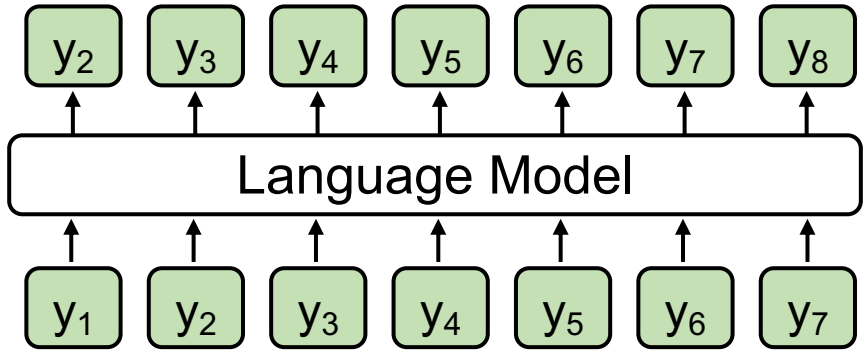
今日著重在 Hugging Face Transformers

https://huggingface.co/docs

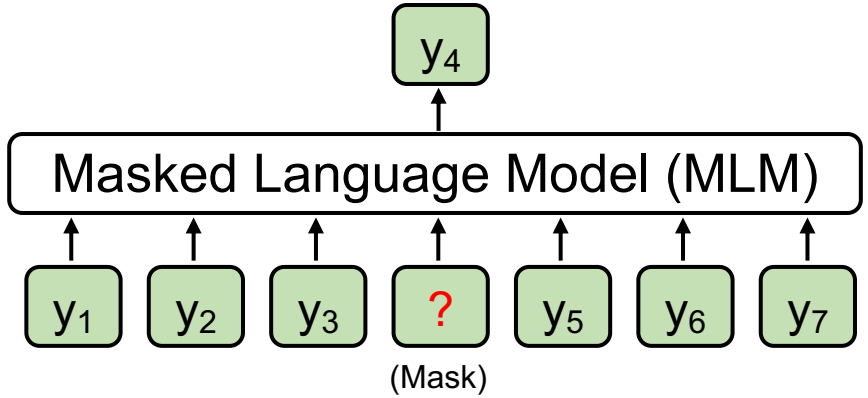


Language Model vs. Masked Language Model

Casual Language Model
(E.g., GPT)

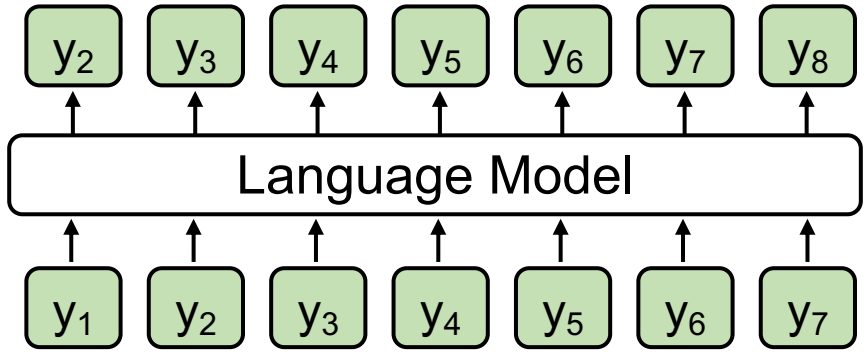


Masked Language Model
(E.g., BERT)

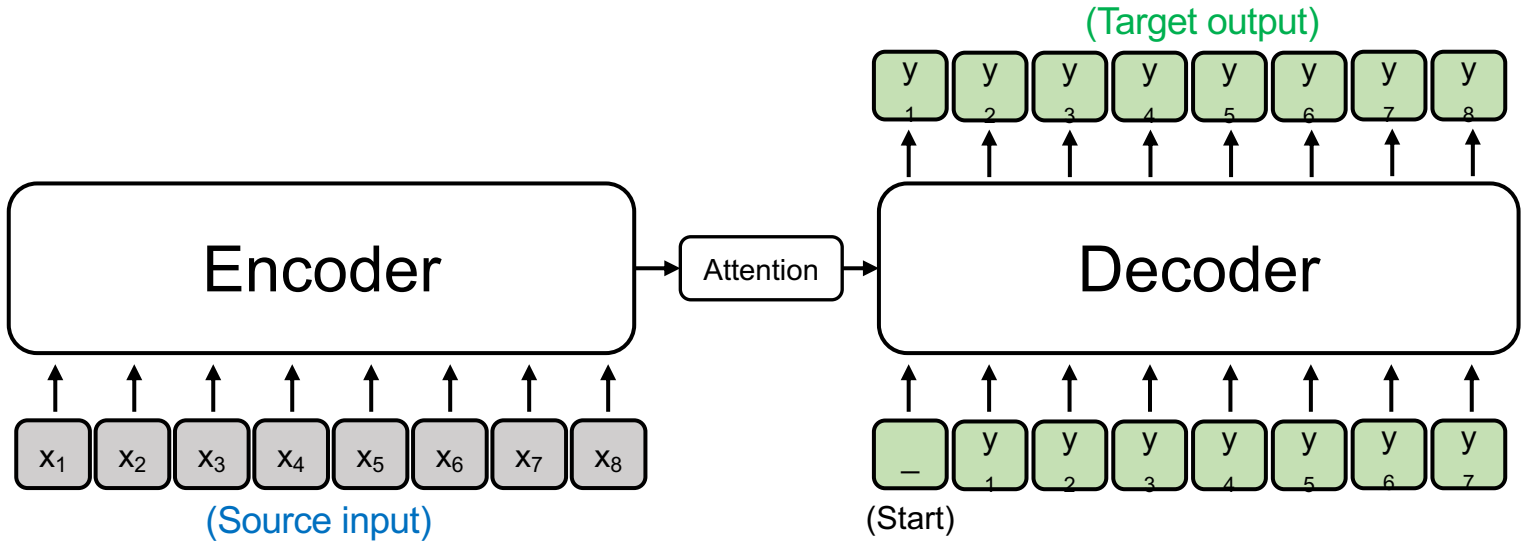


Language Model vs. Seq2seq Model

Casual Language Model
(E.g., GPT)



Sequence to sequence Model
(E.g., T5; Vanilla
Transformers)



Load LCSTS via Hugging Face Dataset

- Hugging Face dataset link

```
raw_train = load_dataset(
   "hugcyp/LCSTS", split="train", cache_dir="./cache/"
).to_list()
raw_val = load_dataset(
   "hugcyp/LCSTS", split="validation", cache_dir="./cache/"
).to_list()
```

Sometimes checking the Hugging Face dataset is slow, it will be faster if we transform the dataset object into a list using `.to_list()`.

Let’s try it

1. 用 load_dataset() 把 train 和 validation 分開讀進來
2. 把 dataset 轉成 pandas dataframe，觀察 1~2 筆 samples

Auto Classes (models, tokenizers)

- In many cases, the architecture you want to use **can be guessed** from the name or the path of the pretrained model you are supplying to the from_pretrained method.
- AutoClasses are here to do this job for you so that you automatically retrieve the relevant model given the name/path to the pretrained weights/config/vocabulary:
  - Ex: model = AutoModel.from_pretrained('bert-base-cased') will create an instance of the BERT base model (cased).

Generative Models in Auto Classes

Casual Language Model (E.g., GPT, Llama)	AutoModelForCausalLM (link) GPT2LMHeadModel (link) LlamaForCausalLM (link)
Masked Language Model (E.g., BERT)	AutoModelForMaskedLM (link) BertForMaskedLM (link)
Sequence to sequence Model (E.g., T5; Vanilla Transformers)	AutoModelForSeq2SeqLM (link) T5ForConditionalGeneration (link)

Outline

- GPT-2 (native PyTorch)
- ~~T5 (Hugging Face Dataset and Trainer)~~

Load the Tokenizer and the Model

- Hugging Face model link

```
model_name = "uer/gpt2-chinese-cluecorpussmall"
tokenizer = AutoTokenizer.from_pretrained(
    model_name,
    padding_side="left”,
)
tokenizer.add_special_tokens({"eos_token": "<|endoftext|>"})
model.resize_token_embeddings(len(tokenizer))
```

Optional

Left padding

In text generation, sometimes we set padding_side='left', when loading tokenizer.

If padding is at the end of the prompt, new tokens may be generated after the padding, which is illogical.

<s>	Recite	the	first	law	<s>	<pad>	<pad>	<pad>	
<s>	How	are	you	<s>	<pad>	<pad>	<pad>	<pad>	
<s>	Who	is	the	first	president	of	U.S.	<s>	

Left-padding

Aligning the new tokens.

<pad>	<pad>	<pad>	<s>	Recite	the	first	law	<s>	
<pad>	<pad>	<pad>	<pad>	<s>	How	are	you	<s>	
<s>	Who	is	the	first	president	of	U.S.	<s>	

About Padding

	Fine-tuning	Test time (inference)
策略一	Left padding + set [PAD] as -100 in labels (this tutorial)	Left padding (this tutorial)
策略二	Right padding + set [PAD] as -100 in labels	Use test_batch_size as 1

- Fine-tuning 的策略一可搭配 Test time 的任一策略
- Fine-tuning 的策略二只建議搭配 Test time 的策略二

Set [PAD] as -100 in labels

https://github.com/huggingface/transformers/blob/main/src/transformers/models/gpt2/modeling_gpt2.py#L1264-L1267

r"""

labels (`torch.LongTensor` of shape `(batch_size, sequence_length)`, *optional*):

Labels for language modeling. Note that the labels **are shifted** inside the model, i.e. you can set `labels = input_ids` Indices are selected in `[-100, 0, ..., config.vocab_size]` All labels set to `-100` are ignored (masked), the loss is only computed for labels in `[0, ..., config.vocab_size]`

"""

Load the Tokenizer and the Model

- Hugging Face model link

```
model_name = "uer/gpt2-chinese-cluecorpussmall"
tokenizer = AutoTokenizer.from_pretrained(
    model_name,
    padding_side="left”,
)
tokenizer.add_special_tokens({"eos_token": "<|endoftext|>"})
model.resize_token_embeddings(len(tokenizer))
```

Increasing the size will add newly initialized vectors at the end.
Reducing the size will remove vectors from the end. (link)

resize_token_embeddings

- 新增 embedding 的情況：
  - 新增的 embedding 還沒有被訓練過
  - 透過 torch 的 nn.Embedding 來新增 token 的向量
  - https://github.com/huggingface/transformers/blob/v4.51.3/src/transformers/modeling_utils.py#L2802-L2807
  - torch 的 nn.Embedding 的初始化範圍是 normal distribution 0到1之間

Let’s try it

1. 載入 tokenizer
2. 測試句子：「今天天氣真好」，print 出 tokenized tokens

Create the PyTorch Dataset

- You can use the Hugging Face dataset without building a PyTorch dataset class.
- However, language generation is complicated. We suggest building your first code with native PyTorch.

```python
class LCSTSDataset(torch.utils.data.Dataset):
    def __init__(self, raw_data) -> None:
        super().__init__()
        self.data = raw_data
        self.token_replacement = [
            ["：", ":"],
            ["，", ","],
            ["“”", '"'],
            ["”“", '"'],
            ["？", "?"],
            ["……", "..."],
            ["！", "!"],
        ]

    def __getitem__(self, index):
        d = self.data[index]
        # Substitute some full-width punctuations with half-width ones
        for k in d:
            for tok in self.token_replacement:
                d[k] = d[k].replace(tok[0], tok[1])
        return d

    def __len__(self):
        return len(self.data)
```

To prevent out-of-vocabulary tokens from being transformed into [UNK]

Run:

```
train_set = LCSTSDataset(raw_train)
val_set = LCSTSDataset(raw_val)
```

Create the PyTorch DataLoader for Data Batching

1. Input data for training (source + summary)
2. Labels (auto-regressive; thus, basically input_ids themselves)
3. Input data for predictions (source only)

Overview

```
1  def collate_fn(batch):
2      complete_text = [
3          f"[CLS]{example['text']}[SEP]{example['summary']}<|endoftext|>"
4          for example in batch
5      ]
6      complete_text = tokenizer(
7  1       complete_text,
8          padding=True,
9          truncation=True,
10         return_tensors="pt",
11         add_special_tokens=False,
12     )
13     # Set label padding tokens to -100 for loss masking
14     labels = torch.where(
15         condition=complete_text.input_ids != tokenizer.pad_token_id,
16         input=complete_text.input_ids,
17 2       other=-100,
18     )
19     complete_text["labels"] = labels
20     complete_text = {k: complete_text[k].to(device) for k in complete_text}
21
22     infer_text = [example["text"] for example in batch]
23     infer_text = tokenizer(
24         infer_text,
25 3       padding=True,
26         truncation=True,
27         return_tensors="pt",
28     )
29     infer_text = {k: infer_text[k].to(device) for k in infer_text}
30     return complete_text, infer_text
```

Create the PyTorch DataLoader for Data Batching: 1. Input Data for Training

```
1   def collate_fn(batch):
2       complete_text = [
3           f"[CLS]{example['text']}[SEP]{example['summary']}<|endoftext|>"
4           for example in batch
5       ]
6       complete_text = tokenizer.batch_encode_plus(
7           complete_text,
8           padding=True,
9           truncation=True,
10          return_tensors="pt",
11          add_special_tokens=False,
12      )
```

You can omit the [CLS] token before fine-tuning!

uer/gpt2-chinese-cluecorpussmall uses BertTokenizer

https://huggingface.co/uer/gpt2-chinese-cluecorpussmall/blob/main/config.json#L26

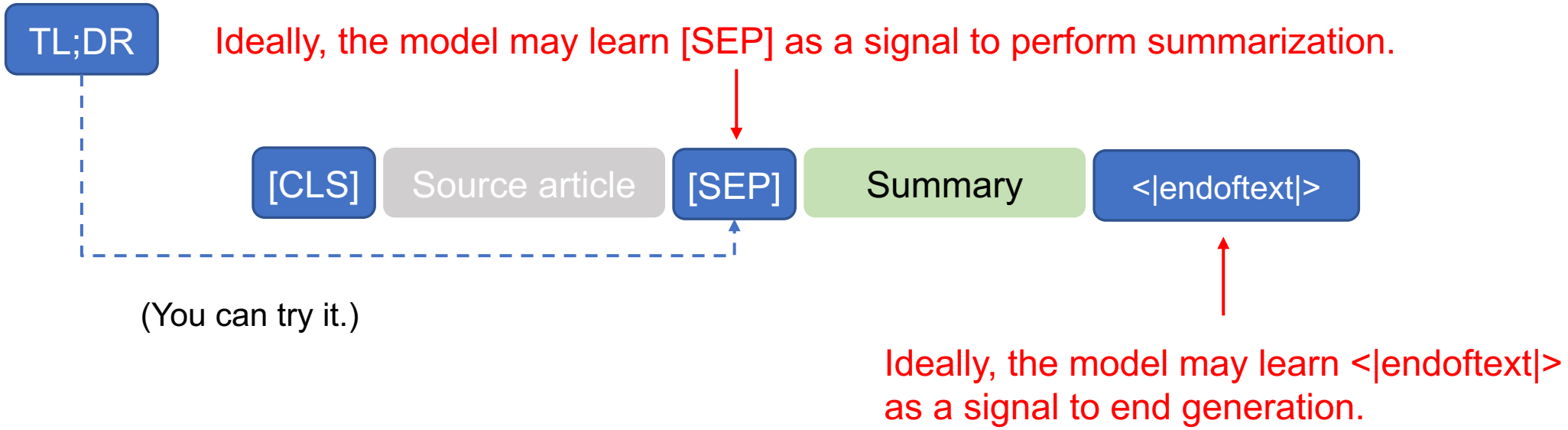
```
20    "task_specific_params": {
21      "text-generation": {
22        "do_sample": true,
23        "max_length": 320
24      }
25    },
26    "tokenizer_class": "BertTokenizer",
27    "vocab_size": 21128
```

https://huggingface.co/ckiplab/gpt2-base-chinese/blob/main/config.json#L32

```
26    "task_specific_params": {
27      "text-generation": {
28        "do_sample": true,
29        "max_length": 50
30      }
31    },
32    "tokenizer_class": "BertTokenizerFast",
33    "vocab_size": 21128
```

English GPT-2 for Text Summarization[2]

> To induce summarization behavior, we can add the text TL;DR: after the article [2]



[2] Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). Language models are unsupervised multitask learners. OpenAI blog, 1(8), 9.

Create the PyTorch DataLoader for Data Batching: 2. Labels (auto-regressive)

(We are still at the `collate_fn` function.)

```
13      # Set label padding tokens to -100 for loss masking
14      labels = torch.where(
15          condition=complete_text.input_ids != tokenizer.pad_token_id,
16          input=complete_text.input_ids,
17          other=-100,
18      )
19      complete_text["labels"] = labels
20      complete_text = {k: complete_text[k].to(device) for k in complete_text}
```

this is sentence a	[PAD]	[PAD]	this	is	sentence	a
labels	-100	-100	1212	318	6827	317
this is is is sentence b	this	is	is	is	sentence	b
labels	1212	318	318	318	6827	347

GPT2LMHeadModel shifts logits and labels

https://github.com/huggingface/transformers/blob/main/src/transformers/models/gpt2/modeling_gpt2.py#L1300-L1301

Shape of lm_logits: [batcg_size, seq_length, hidden_size]

```
1296            if labels is not None:
1297                # move labels to correct device to enable model parallelism
1298                labels = labels.to(lm_logits.device)
1299                # Shift so that tokens < n predict n
1300                shift_logits = lm_logits[..., :-1, :].contiguous()  :-1 代表取到倒數第二個
1301                shift_labels = labels[..., 1:].contiguous()         1: 代表答案後移一格
1302                # Flatten the tokens
1303                loss_fct = CrossEntropyLoss()
1304                loss = loss_fct(shift_logits.view(-1, shift_logits.size(-1)), shift_labels.view(-1))
```

.contiguous():
https://youtu.be/FGJSGAt_9Ks?si=GMxxS2Oi742e2BhI&t=1261

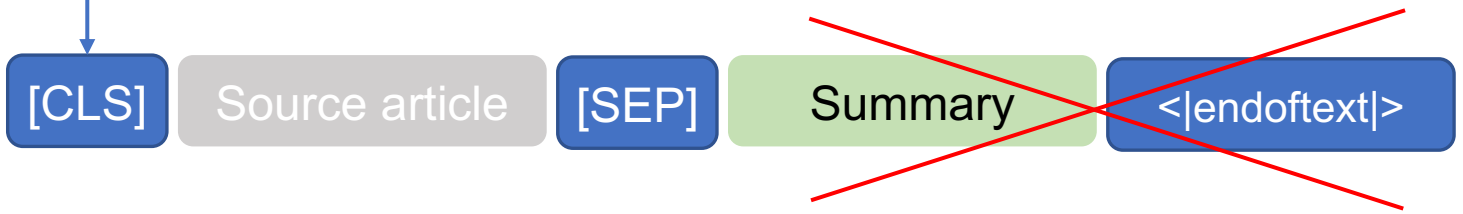
Create the PyTorch DataLoader for Data Batching: 3. Input Data for Predictions

(We are still at the `collate_fn` function.)

We will use `infer_text` during evaluations.

```
22      infer_text = [example["text"] for example in batch]
23      infer_text = tokenizer.batch_encode_plus(
24          infer_text,
25          padding=True,
26          truncation=True,
27          return_tensors="pt",
28      )
29      infer_text = {k: infer_text[k].to(device) for k in infer_text}
30      return complete_text, infer_text
```

If you omit the [CLS] token during fine-tuning, then you don't need to add [CLS] before evaluations!



Create the PyTorch DataLoader for Data Batching: 1. Input Data for Training [Recap]

```
1   def collate_fn(batch):
2       complete_text = [
3           f"[CLS]{example['text']}[SEP]{example['summary']}<|endoftext|>"
4           for example in batch
5       ]
6       complete_text = tokenizer.batch_encode_plus(
7           complete_text,
8           padding=True,
9           truncation=True,
10          return_tensors="pt",
11          add_special_tokens=False,
12      )
```

You can omit the [CLS] token before fine-tuning!

Create the PyTorch DataLoader for Data Batching

```
1   train_loader = torch.utils.data.DataLoader(
2       train_set,
3       batch_size=TRAIN_BATCH_SIZE,
4       shuffle=True,
5       collate_fn=collate_fn,
6   )
7   val_loader = torch.utils.data.DataLoader(
8       val_set,
9       batch_size=VAL_BATCH_SIZE,
10      shuffle=False,
11      collate_fn=collate_fn,
12  )
```

Prevent a model from overfitting on data order

We don’t need to use the test set because there is no public test labels (summaries).

See https://huggingface.co/datasets/hugcyp/LCSTS/viewer/default/test

Set up Optimizer

```
optimizer = torch.optim.AdamW(model.parameters(), lr=LR)
```

- **optimizers** (`Tuple[torch.optim.Optimizer, torch.optim.lr_scheduler.LambdaLR]`, *optional*, defaults to `(None, None)`) — A tuple containing the optimizer and the scheduler to use. Will default to an instance of AdamW on your model and a scheduler given by get_linear_schedule_with_warmup() controlled by `args`.

Trainer

Trainer

Seq2SeqTrainer

TrainingArguments

Seq2SeqTrainingArguments

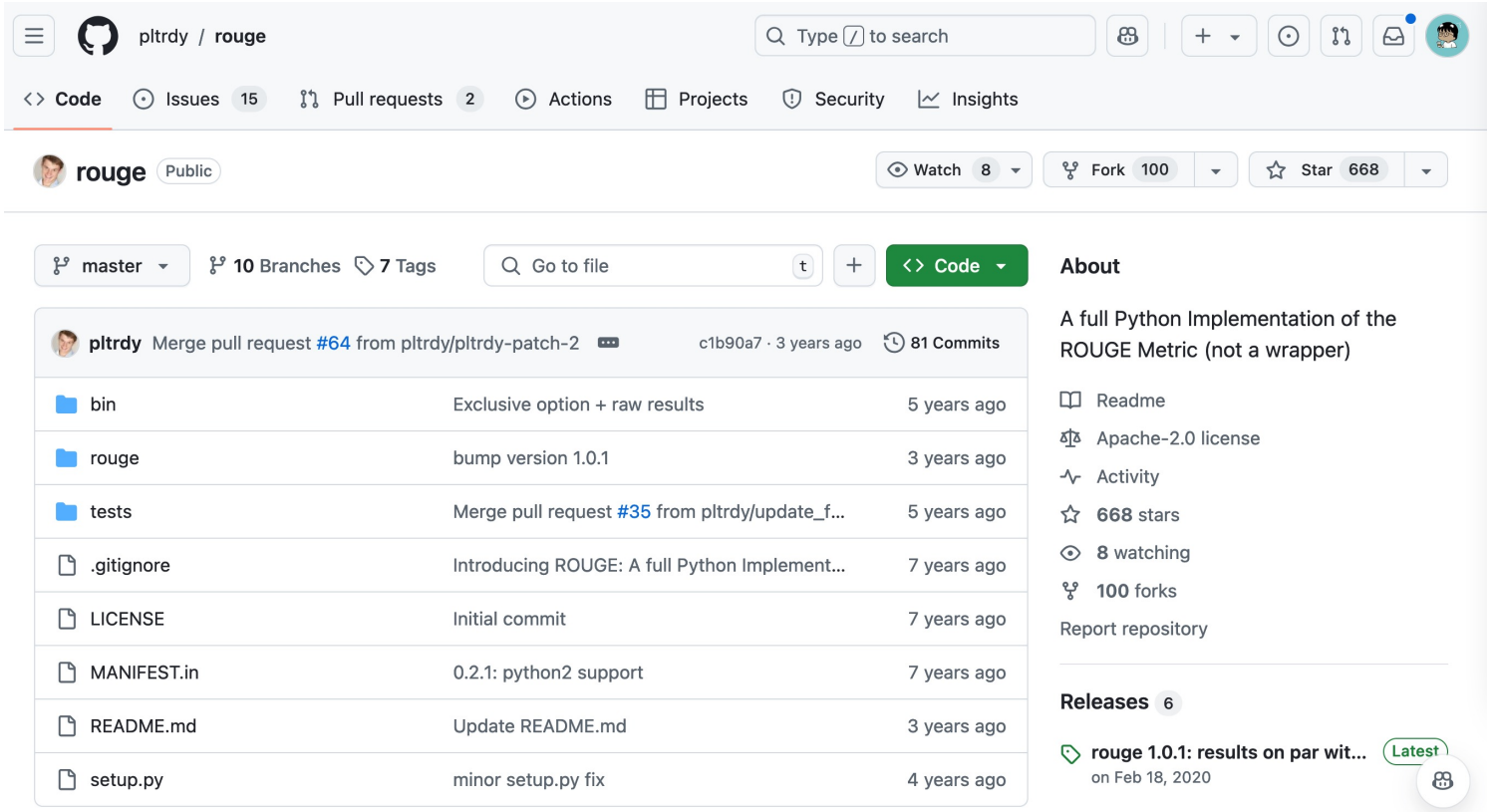
https://pytorch.org/docs/stable/generated/torch.optim.AdamW.html
https://huggingface.co/docs/transformers/main_classes/trainer

Let’s try it

1. 開 train 一下模型

Set up Evaluation Metric

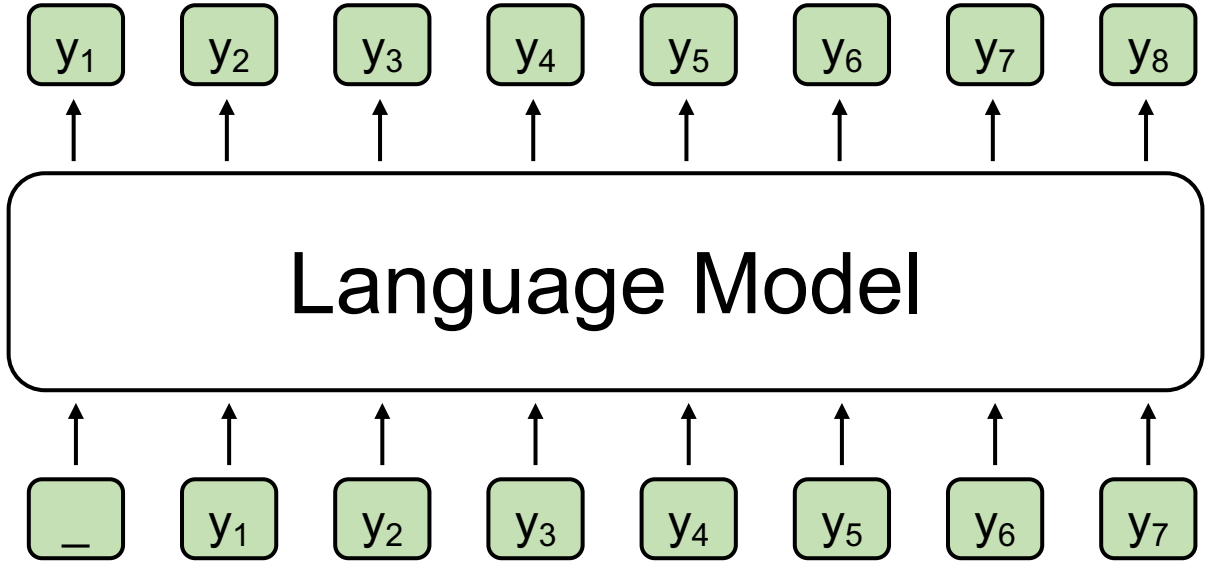
```
rouge_metric = Rouge()
```



https://github.com/pltrdy/rouge

Generating step-by-step

Casual Language Model (E.g., GPT)



- Training time: update the model through hidden states
- Test time: perform decoding for each step

Decoding strategies

- Popular decoding strategies:
  - Greedy decoding
  - Beam search
  - Top-k sampling
  - Top-p sampling

https://huggingface.co/blog/how-to-generate

How to let a model generate step-by-step?

- We can use model.generate()
- There are two main advantages of model.generate():
  1. We don’t need to write a decoding loop on our own.
  2. We don’t need to implement decoding strategies by ourselves.

[Useful!!] https://huggingface.co/blog/how-to-generate

Pseudo code of a decoding loop

```
for step in range(max_length):
    # Forward pass through the model
    outputs = model.decoder(
        input_ids=decoder_input_ids,
        encoder_outputs=encoder_outputs
    )

    # Take the last hidden state's logits
    logits = outputs.logits[-1]  # shape: (vocab_size,)

    # Apply softmax to get probabilities (optional, since argmax is invariant)
    probs = softmax(logits)

    # Select the token with the highest probability (greedy decoding)
    next_token_id = argmax(probs)

    # Append the predicted token to the decoder inputs
    decoder_input_ids.append(next_token_id)

    # If EOS token is generated, stop decoding
    if next_token_id == EOS_TOKEN_ID:
        break
```

Perform Evaluations

Input: [CLS] Source article [SEP]

Expected output of `model.generate()`:

[CLS] Source article [SEP] Summary <|endoftext|>

Gold: Summary

Perform Evaluations

Output Part

```python
def do_evaluate(
    tokenizer,
    model,
    validation_loader,
    rouge_metric,
    inner_check=False,
):
    pbar = tqdm(validation_loader)
    pbar.set_description(f"Evaluating")

    predictions = []
    references = []
    count = 0
    for ground_truth, inputs in pbar:
        output = [
            s.split("[SEP]")[1].replace(" ", "").split("<|endoftext|>")[0]
            for s in tokenizer.batch_decode(
                model.generate(
                    **inputs,
                    max_new_tokens=200,
                    pad_token_id=tokenizer.eos_token_id,
                )
            )
        ]
```

Expected output of `model.generate()`:

[CLS] | Source article | [SEP] | Summary | <|endoftext|>

Decoding 後的處理 (接續上一頁)

```
output = [
    s.split("[SEP]")[1].replace(" ", "").split("<|endoftext|>")[0]
    for s in tokenizer.batch_decode(
        model.generate(
            **inputs,
            max_new_tokens=200, # Maximum number of tokens to generate
            pad_token_id=tokenizer.eos_token_id,
        )
    )
]
```

s: [CLS] 中國 足協 今 日 宣布 與 國際 足聯 達成 合作 協議 [SEP] 中國 足協 與 國際 簽署 協議 <|endoftext|>

s.split("[SEP]")[1]: 中國 足協 與 國際 足聯 簽署 協議 <|endoftext|>

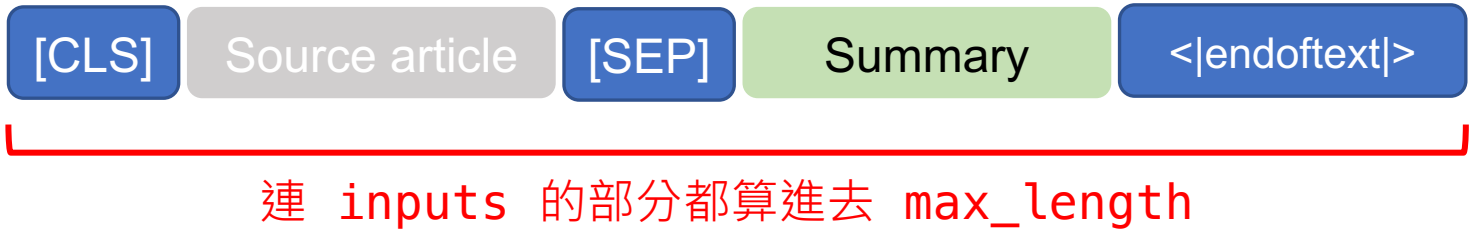
.replace(" ", ""): 中國足協與國際足聯簽署協議<|endoftext|>

.split("<|endoftext|>")[0]:中國足協與國際足聯簽署協議

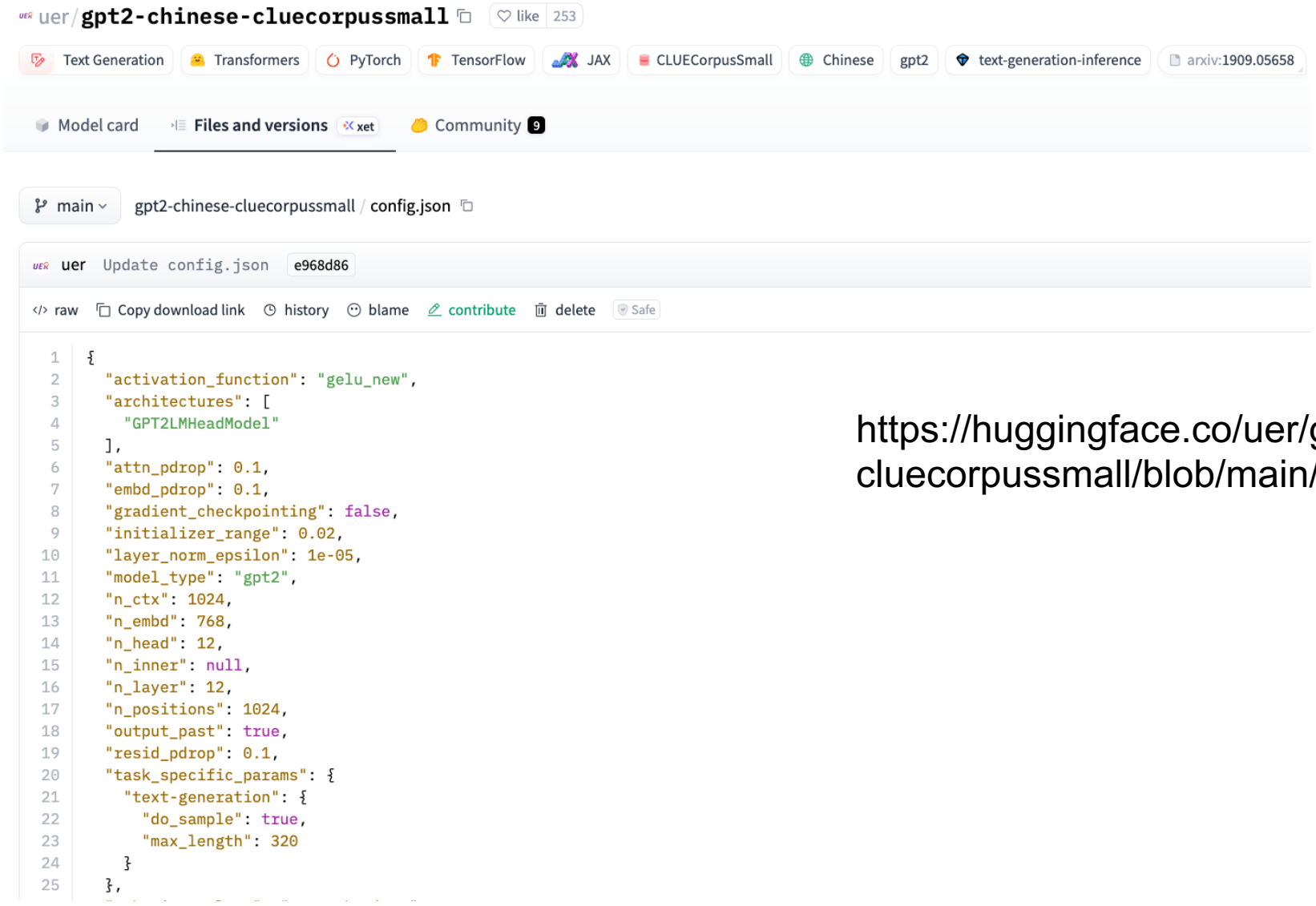
通常我們會設置 max_new_tokens

```
output = [
    s.split("[SEP]")[1].replace(" ", "").split("<|endoftext|>")[0]
    for s in tokenizer.batch_decode(
        model.generate(
            **inputs,
            max_new_tokens=200, # Maximum number of tokens to generate
            pad_token_id=tokenizer.eos_token_id,
        )
    )
]
```

如果你不設 `max_new_tokens`，那模型 `outputs` 就會受到 `max_length` 控制：



查看 model config 最簡單的方式



https://huggingface.co/uer/gpt2-chinese-cluecorpussmall/blob/main/config.json

在 batch 生成時要用 EOS 去替代 Padding

```
output = [
    s.split("[SEP]")[1].replace(" ", "").split("<|endoftext|>")[0]
    for s in tokenizer.batch_decode(
        model.generate(
            **inputs,
            max_new_tokens=200, # Maximum number of tokens to generate
            pad_token_id=tokenizer.eos_token_id,
        )
    )
]
```

原因：GPT-2 沒有 [PAD] token!

假設一個 batch 在生成時：

Sent1: 中國 足球 協會 今天 ... <|endoftext|>

Sent2: 美國 <|endoftext|> <|endoftext|> <|endoftext|> <|endoftext|> <|endoftext|>

do_evaluate 的 targets

```
25          targets = [
26              s.split("[SEP]")[1].replace(" ", "").replace("<|endoftext|>", "")
27              for s in tokenizer.batch_decode(ground_truth["input_ids"])
28          ]
29          assert len(output) == len(targets)
30          output = [" "] if output == [""] else output
31          predictions.extend([" ".join(jieba.lcut(o)) for o in output])
32          references.extend([" ".join(jieba.lcut(t)) for t in targets])
33          count += 1
34          if count > 100 and inner_check:
35              break
36
37      score = rouge_metric.get_scores(predictions, references, avg=True)
38      if inner_check:
39          print("Validation using 100 examples: ", score)
40      else:
41          print(score)
42
43      return score, predictions, references
```

同「Decoding後的處理」那頁

ground_truth (對應 collate_fn 的 complete_text):

do_evaluate 的 predictions

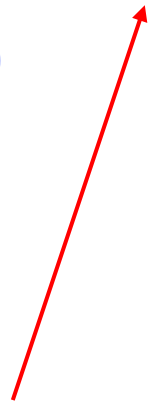
```
25          targets = [
26              s.split("[SEP]")[1].replace(" ", "").replace("<|endoftext|>", "")
27              for s in tokenizer.batch_decode(ground_truth["input_ids"])
28          ]
29          assert len(output) == len(targets)
30          output = [" "] if output == [""] else output
31          predictions.extend([" ".join(jieba.lcut(o)) for o in output])
32          references.extend([" ".join(jieba.lcut(t)) for t in targets])
33          count += 1
34          if count > 100 and inner_check:
35              break
36
37      score = rouge_metric.get_scores(predictions, references, avg=True)
38      if inner_check:
39          print("Validation using 100 examples: ", score)
40      else:
41          print(score)
42
43      return score, predictions, references
```

outputs 和 targets 現在都是文字

使用結巴斷詞後，才會計算 ROUGE (正確答案和預測的重疊率)

do_evaluate 的 score

```
25            targets = [
26                s.split("[SEP]")[1].replace(" ", "").replace("<|endoftext|>", "")
27                for s in tokenizer.batch_decode(ground_truth["input_ids"])
28            ]
29            assert len(output) == len(targets)
30            output = [" "] if output == [""] else output
31            predictions.extend([" ".join(jieba.lcut(o)) for o in output])
32            references.extend([" ".join(jieba.lcut(t)) for t in targets])
33            count += 1
34            if count > 100 and inner_check:
35                break
36
37        score = rouge_metric.get_scores(predictions, references, avg=True)
38        if inner_check:
39            print("Validation using 100 examples: ", score)
40        else:
41            print(score)
42
43        return score, predictions, references
```



avg=True: perform averaging for all the tested instances

完整的 Training Loop

```python
step_i = 0
for epoch in range(NUM_EPOCHS):
    pbar = tqdm(train_loader)
    pbar.set_description(f"Training epoch [{epoch+1}/{NUM_EPOCHS}]")
    for inputs, _ in pbar:
        optimizer.zero_grad()
        loss = model(**inputs).loss
        loss.backward()
        optimizer.step()
        pbar.set_postfix(loss=loss.item())
        writer.add_scalar("Loss/train", loss.item(), step_i)

        if step_i % 1000 == 0 and step_i != 0:
            score, pres, refs = do_evaluate(
                tokenizer=tokenizer,
                model=model,
                validation_loader=val_loader,
                rouge_metric=rouge_metric,
                inner_check=True,
            )
            print(f"Rouge scores on step{step_i} of epoch {epoch}:", score)
            print("Predictions:", pres[:5])
            writer.add_scalar("Rouge-1/val", score["rouge-1"]["f"], step_i)
            writer.add_scalar("Rouge-2/val", score["rouge-2"]["f"], step_i)
        step_i += 1
```

Let’s try it

1. 看一下你自己的 Wandb

T5 [4]

[4] Raffel, Colin, et al. "Exploring the limits of transfer learning with a unified text-to-text transformer." *Journal of machine learning research* 21.140 (2020): 1-67.

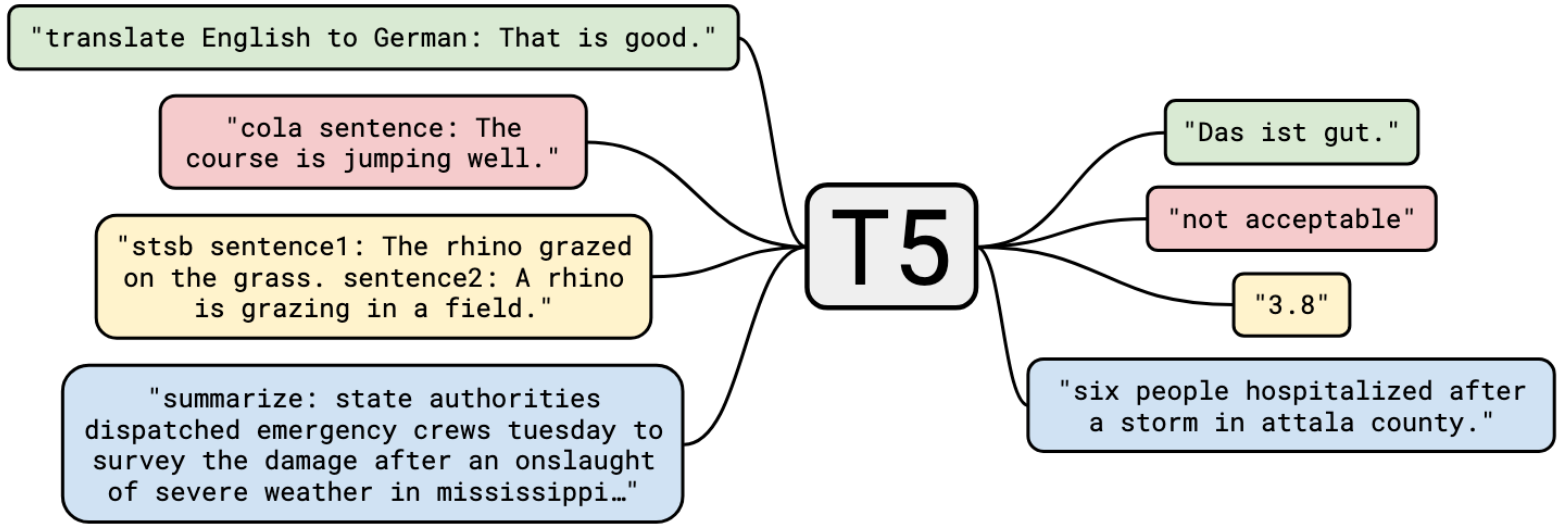
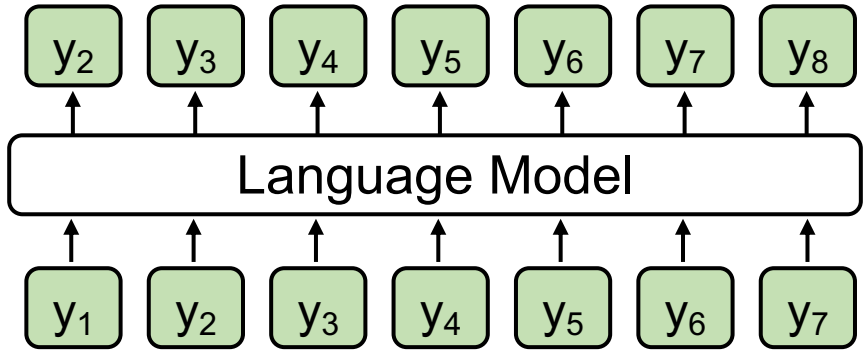


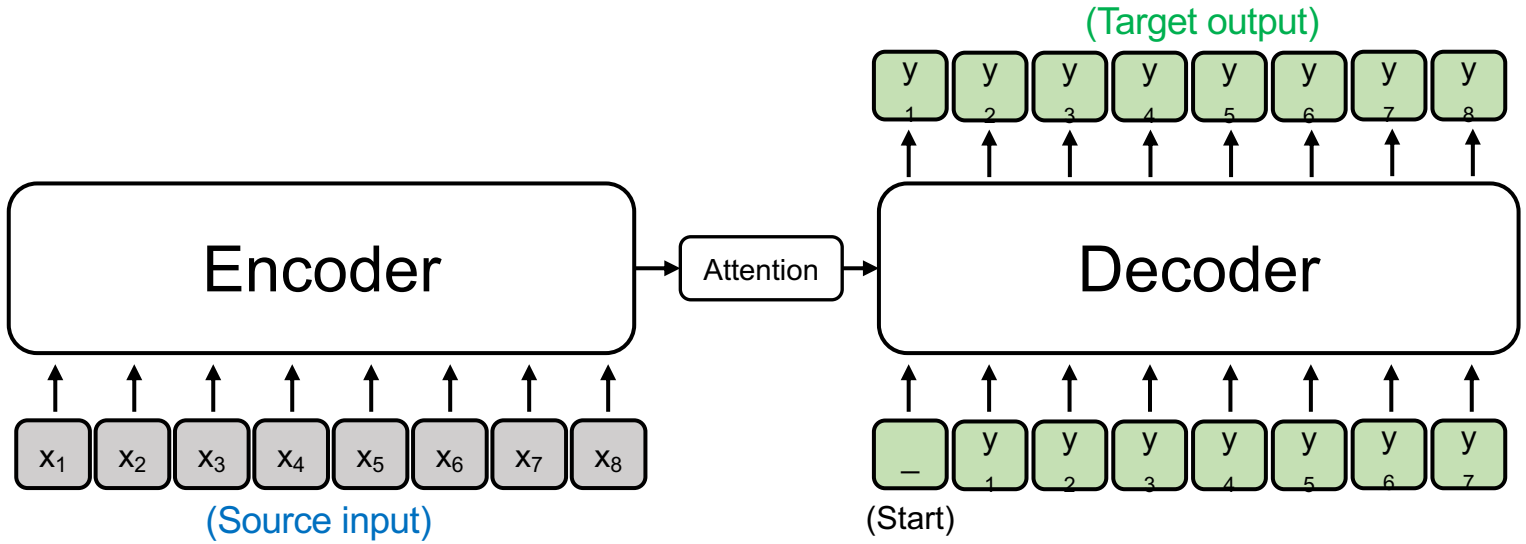
Figure 1: A diagram of our text-to-text framework. Every task we consider—including translation, question answering, and classification—is cast as feeding our model text as input and training it to generate some target text. This allows us to use the same model, loss function, hyperparameters, etc. across our diverse set of tasks. It also provides a standard testbed for the methods included in our empirical survey. "T5" refers to our model, which we dub the "**T**ext-**t**o-**T**ext **T**ransfer **T**ransformer".

Language Model vs. Seq2seq Model

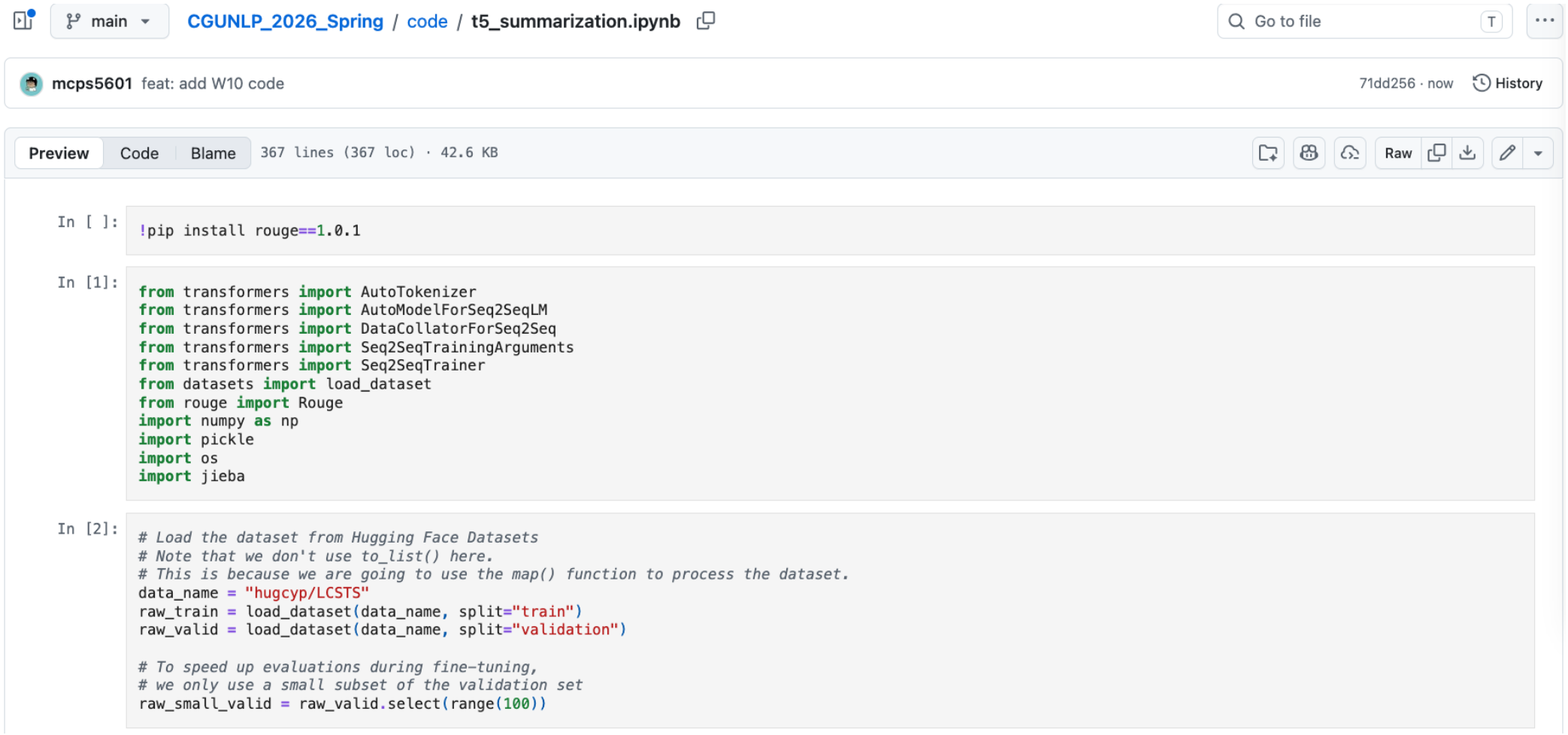
Casual Language Model
(E.g., GPT)



Sequence to sequence Model
(E.g., T5; Vanilla
Transformers)



See `code/t5_summarization.ipynb`



Thank you!

Instructor: 林英嘉

✉ yjlin@cgu.edu.tw

TA: 林宣辰

✉ b1128015@cgu.edu.tw